一、Linux 常用基础指令（速查 + 典型用法）

1) 文件/目录（最常用）

```
pwd
ls [-alh]
cd DIR | cd .. | cd - | cd ~
mkdir [-p] DIR
touch FILE
cp [-r] SRC DST
mv SRC DST
rm [-rf] PATH
```

2) 查看文件（文本/日志）

```
cat FILE
less FILE            # /pattern 搜索, n 下一个, q 退出
head [-n N] FILE
tail [-n N] FILE
tail -f LOGFILE      # 跟踪日志
```

3) 权限（会考/会踩坑）

```
ls -l
chmod 755 FILE       # rwxr-xr-x
chmod u+x FILE
chown user:group FILE
```

权限位：`r(4) w(2) x(1)`，三组分别是 **所有者/同组/其他人**。目录的 `x` 表示“可进入/可遍历”。

4) 进程/系统

```
ps
ps aux | grep NAME
top
kill PID
kill -9 PID
df -h
du -sh DIR
free -h
uname -a
```

5) 查找

```
which CMD
whereis CMD
find PATH -name 'PATTERN'
```

二、正则表达式（grep/sed/awk 都会用）

1) 最常见规则（背这几个就够用大部分题）

- `.`：任意单字符（不含换行）
- `*`：前一个元素重复 0 次或多次
- `+`：前一个元素重复 1 次或多次（通常需要 **扩展正则**）
- `?`：前一个元素重复 0 或 1 次（通常需要扩展正则）
- `^`：行首
- `$`：行尾
- `[abc]`：集合，匹配 a/b/c 任一
- `[a-z]`：范围
- `[^a-z]`：取反集合
- `\{m,n\}`：重复次数（很多工具里需要转义；扩展正则常可写 `{m,n}`）
- `(...)` 分组、`|` 或（多为扩展正则）

2) 基本正则 vs 扩展正则（重点区别）

- **基本正则（BRE）**：`grep`、`sed` 默认常用
  - `+ ? | ( ) { }` 通常 **需要反斜杠** 才有特殊含义
- **扩展正则（ERE）**：`grep -E`、`awk` 默认更接近 ERE
  - `+ ? | ( ) { }` 通常 **不需要反斜杠**

3) 常见例子（直接套）

```
^$              # 空行
^#              # 以 # 开头
\.c$            # 以 .c 结尾（点要转义）
[0-9]+          # 一段数字（ERE）
error|warn      # error 或 warn（ERE）
```

三、grep（查找/过滤，最常配管道）

1) 用法总览

```
grep [OPTIONS] PATTERN [FILE...]
cmd | grep [OPTIONS] PATTERN
```

2) 高频参数（打印版建议背）

- `-n`：显示行号
- `-i`：忽略大小写
- `-v`：反选（不匹配）
- `-r`：递归目录
- `-E`：扩展正则（ERE）
- `-F`：固定字符串（不当正则，速度快）
- `-o`：只输出匹配到的片段
- `-c`：只输出匹配行数

3) 典型题型模板

```
grep -n "str" file
grep -nE "a|b|c" file
grep -r --line-number "TODO" .
ps aux | grep -v grep | grep nginx
```

四、awk（按列处理 + 条件 + 统计）

1) 用法总览

```
awk 'PATTERN { ACTION }' FILE
awk -F':' '...' FILE          # 指定分隔符
cmd | awk '...'               # 管道输入
```

2) 内置变量（特别常用）

- `$0`：整行
- `$1..$N`：第 1..N 列
- `NF`：本行列数
- `NR`：当前行号（全局）
- `FS`：输入分隔符（等价 `-F`）
- `OFS`：输出分隔符（默认空格）

3) 常用动作模板

```
awk '{print $1, $3}' file
awk -F: '{print $1, $3}' /etc/passwd
awk 'NR==1{print; next} {print NR ":" $0}' file
awk '$2>=60 && $2<=80 {print $1, $2}' score.txt
awk '{sum+=$2} END{print sum}' data.txt
awk '{cnt[$1]++} END{for(k in cnt) print k, cnt[k]}' file
```

4) 常见坑

- awk 默认按“空白”分列；如果文本是 `a:b:c` 这种，一定要 `-F:`
- `print $1,$2` 中间自动输出 OFS（默认空格），不是逗号

五、sed（流编辑器：替换、删行、抽取范围）

1) 用法总览

```
sed 'SCRIPT' FILE
sed -n 'SCRIPT' FILE            # 关闭默认输出，只输出你指定的
sed -i 'SCRIPT' FILE            # 就地修改（谨慎，建议先不加 -i 验证）
```

2) 替换（最常考）

```
sed 's/old/new/' file           # 每行第 1 个
sed 's/old/new/g' file          # 每行所有
sed 's/old/new/2' file          # 每行第 2 个
sed 's/old/new/2g' file         # 从第 2 个到行尾
```

范围/条件替换：

```
sed '5s/old/new/' file          # 只改第 5 行
sed '2,4s/old/new/g' file       # 只改 2~4 行
sed '/pat/s/old/new/g' file     # 只改匹配 pat 的行
```

3) 删除/抽取

```
sed '/^$/d' file                # 删空行
sed '/DEBUG/d' file             # 删包含 DEBUG 的行
sed -n '10,20p' file            # 打印 10~20 行
```

4) 常见坑

- `-i` 会直接改文件，建议先 `sed '...' file | less` 确认，再加 `-i`
- 替换内容里如果包含 `/`，可以换分隔符：`s#old/new#X#g`

六、Shell 脚本（bash）：结构模板 + 分支/循环 + 易错点

1) 脚本骨架（建议固定模板）

```bash
#!/usr/bin/env bash
set -e

# 用法: bash script.sh arg1 arg2 ...
```

（`set -e`：任一命令失败就退出，写实验脚本很有用；如果你不想要它可以删）

2) 变量、引用、命令替换（高频坑点）

```bash
name="Alice"          # 等号两边不能有空格
echo "$name"          # 变量建议永远加双引号，避免空格/通配符展开

now=$(date)           # 命令替换
```

参数相关：

```bash
$0   # 脚本名
$1   # 第 1 个参数
$#   # 参数个数
$@   # 所有参数（逐个参数展开，推荐）
$?   # 上个命令退出码（0 成功）
```

3) if 分支（格式直接背）

```bash
if [ 条件 ]; then
  ...
elif [ 条件 ]; then
  ...
else
  ...
fi
```

整数比较：

```bash
[ "$a" -eq 1 ]   # 等于
[ "$a" -ne 1 ]   # 不等于
[ "$a" -gt 1 ]   # 大于
[ "$a" -ge 1 ]   # 大于等于
[ "$a" -lt 1 ]   # 小于
[ "$a" -le 1 ]   # 小于等于
```

字符串比较：

```
[ "$s" = "abc" ]
[ "$s" != "abc" ]
[ -z "$s" ]      # 空字符串
[ -n "$s" ]      # 非空字符串
```

文件测试：

```
[ -e path ]      # 存在
[ -f file ]      # 普通文件
[ -d dir ]       # 目录
```

逻辑组合：

```
[ cond1 ] && [ cond2 ]
[ cond1 ] || [ cond2 ]
```

4) case 分支（多分支更清晰）

```
case "$1" in
  start)
    echo "start"
    ;;
  stop|kill)
    echo "stop"
    ;;
  *)
    echo "usage: $0 {start|stop}"
    exit 1
    ;;
esac
```

5) 循环（for/while）模板

```
for i in 1 2 3; do
  echo "$i"
done

for f in *.c; do
  echo "file=$f"
done

i=1
while [ "$i" -le 5 ]; do
```

```
  echo "$i"
  i=$((i+1))
done
```

6) 常见难点/坑总结

- **永远给变量加引号**：`"$1"`、`"$file"`，避免文件名里有空格或通配符 `*` 被展开
- `[` 和 `]` 两边都要有空格：`[ "$a" -eq 1 ]`
- `==` 在 `[` 里并不总是可移植，建议用 `=`
- 重定向覆盖：`>` 覆盖，`>>` 追加
- 管道：`cmd1 | cmd2`，前者标准输出进入后者标准输入

七、Makefile：常用写法 + 难点/坑 + 排错

1) 规则格式（最核心）

```
target: prerequisites
    recipe
```

注意：`recipe` 行必须以 **TAB** 开头（不是空格）。

2) 变量（建议都写）

```
CC := gcc
CFLAGS := -Wall -O2
```

使用变量：

```
$(CC) $(CFLAGS) -c main.c -o main.o
```

3) 自动变量（很常用）

- `$@`：目标名
- `$<`：第一个依赖
- `$^`：所有依赖

4) 推荐的最小可维护模板（可直接抄）

```
CC := gcc
CFLAGS := -Wall -O2

TARGET := app
```

```
OBJS := main.o foo.o

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean
clean:
    rm -f $(TARGET) $(OBJS)
```

5) 多目录常见写法（外层调用子目录）

```
.PHONY: all clean

all:
    $(MAKE) -C code

clean:
    $(MAKE) -C code clean
```

6) Makefile 常见难点/坑（实验里最容易卡的）

- **TAB 问题**：`missing separator` 基本就是 recipe 行没 TAB
- **目标与文件同名**：例如 `clean`，必须 `.PHONY: clean`
- **增量编译依赖写错**：头文件变了却没重新编译 → 把 `.h` 加入依赖（或用自动依赖生成）
- **变量覆盖**：`=` 是递归展开，`:=` 是立即展开（简单项目一般都能用 `:=`）
- **命令回显/静默**：在命令前加 `@` 可以不打印命令本身（看起来更干净）

7) 排错手段（很有用）

```
make -n        # 只打印将要执行的命令，不真正执行
make -B        # 无视时间戳，强制重建
make -C dir    # 在 dir 目录里执行 make
```

八、组合用法（grep/awk/sed + 重定向/管道）

1) 输出行号到文件（常见题型）

```
grep -n "int" file | awk -F: '{print $1}' > result
```

2) 仅输出匹配行号（sed 也能做）

```
sed -n '/int/=' file > result
```